



Python

MIRACLIN

Prepared By Miraclin Technologies

Installing python

- <https://www.python.org/downloads/>
- Choose your desired version based on your OS



Python >>> Downloads >>> Windows



- Click on the required drive to install python and choose necessary settings., Install.
- To test : Cmd ☐ *python*

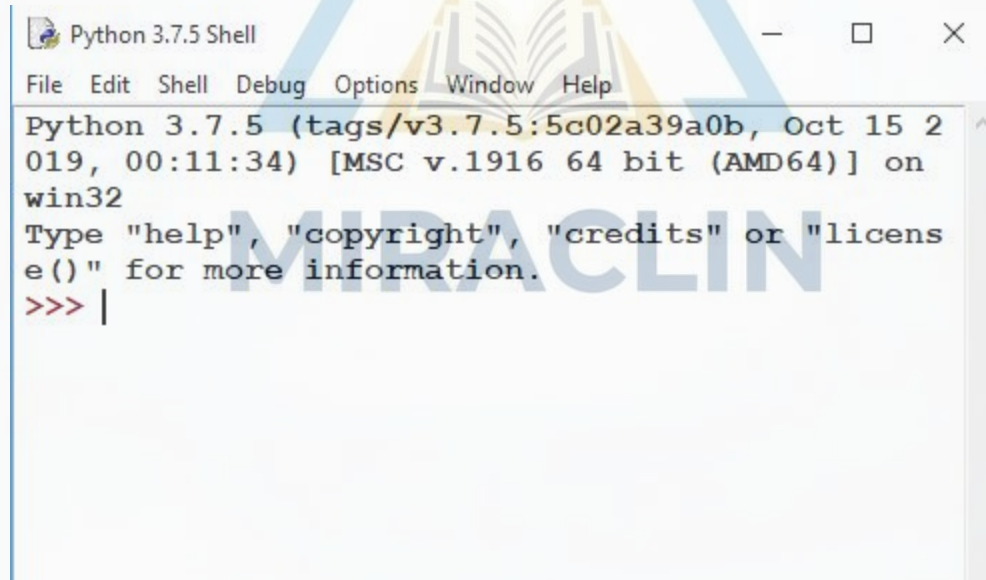
```
Command Prompt - python
Microsoft Windows [Version 10.0.17134.48]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\aldo>python
Python 3.7.4 (default, Aug  9 2019, 18:34:13) [MSC v.1915 64 bit (AMD64)] :: Anaconda, Inc. on win32

Warning:
This Python interpreter is in a conda environment, but the environment has
not been activated. Libraries may fail to load. To activate this environment
please see https://conda.io/activation

Type "help", "copyright", "credits" or "license" for more information.
>>>
```

- Python IDE

A screenshot of a Windows application window titled "Python 3.7.5 Shell". The window has a standard menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area displays the following information: "Python 3.7.5 (tags/v3.7.5:5c02a39a0b, Oct 15 2019, 00:11:34) [MSC v.1916 64 bit (AMD64)] on win32". Below this, it says "Type 'help', 'copyright', 'credits' or 'license()' for more information." and ends with the prompt ">>> |". A large, semi-transparent watermark "MIRACLIN" is overlaid across the center of the image.

```
Python 3.7.5 Shell
File Edit Shell Debug Options Window Help
Python 3.7.5 (tags/v3.7.5:5c02a39a0b, Oct 15 2019, 00:11:34) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```

Python language Basic Constructs

- What is a control flow?
- Sequential flow
- Conditionals
- Looping
- Function calls
- Function definitions, Parameters and arguments to functions Recursive functions



What is Control Flow?

- Sequence in which program components are executed
- top-to-bottom, left-to-right order.
("Sequential flow")

```
print 1  
↓  
print 2  
↓  
print 3; → print 4  
←  
print 5  
↓  
print 6; → print 7; → print 8
```

1
2
3
4
5
6
7
8

Conditionals with if and else, etc.

- Altering sequential flow using conditionals
- A conditional usually starts with the keyword **if**

Example:

```
In [3]: name = input('Name : ')
        if len(name)>4:
            print("Long name")
        else:
            print("Short one")
```

```
Name : Aldrin
Long name
```

- The flow:
- Condition: evaluates True or False
- Print ☐ 'Then clause'
- 'else clause'

Looping

- Another way to alter sequential flow is using loops.
- A loop usually starts with the keyword `for` or `while`.

```
for i in range(0,10):  
    print(i)
```

0
1
2
3
4
5
6
7
8
9

```
for i in range(0,10):  
    print(i*i)
```

0
1
4
9
16
25
36
49
64
81

- The flow:
- The loop variable or the 'index', here its is "i" mentioned with the range.
- The loop body gets executed repeatedly based on the loop criteria.

While loop

- Initializing of loop control flag,
- Loop condition
- The loop body
- Getting executed repeatedly

```
done = False;
while not done:
    x = input('Type a word: ')
    if x == "":
        done = True #get ready to terminate.
        continue #finish the iteration
    word_in = x + " is the word typed"
    print("Execution of While loop: " + word_in)
```

```
Type a word: Hello
Execution of While loop: Hello is the word typed
```

Making Computers Do a Lot of Work

- Loops: Once it is written, it should make the machine repeat the body as long as the condition satisfied.
- Little work from coder ☐ lot work from machine
- Another forms: FUNCTIONS ☐ define a function, and calling it many times. set of statements that take inputs, do some specific computation and produces output.

Functions in Python

- A function can serve as a kind of “building-block” for your program.
- A function may be either built-in, or user-defined, or separately installed.
- The uses may be within the same program or in other programs.

Built-in functions

```
In [17]: max(3,9)
```

```
Out[17]: 9
```

```
In [18]: min(1,7)
```

```
Out[18]: 1
```

```
In [19]: len("Aldrin")
```

```
Out[19]: 6
```

```
In [20]: float("3.4")
```

```
Out[20]: 3.4
```

```
In [21]: int(float("3.4"))
```

```
Out[21]: 3
```

```
In [22]: import math  
math.sin(2.0)
```

```
Out[22]: 0.9092974268256817
```

```
In [23]: math.cos(2.0)
```

```
Out[23]: -0.4161468365471424
```

User-Defined Functions

- Defining a function
- Function body
- Then function calls.,
ie., mycube(4)

```
In [24]: def mycube(x):  
         sqr = x * x;  
         res = sqr * x;  
         return res
```

```
In [25]: mycube(4)
```

```
Out[25]: 64
```

Recursive function

- Recursions can be very useful.
- It's tricky.
- The example, is recursive since it occurs within the body of its own definition

```
In [28]: def fact(n):  
         if n == 0:  
             return 1  
         else:  
             return n * fact(n-1)  
  
         print(fact(0))  
         print(fact(5))
```

```
1  
120
```


- **Advantages:**

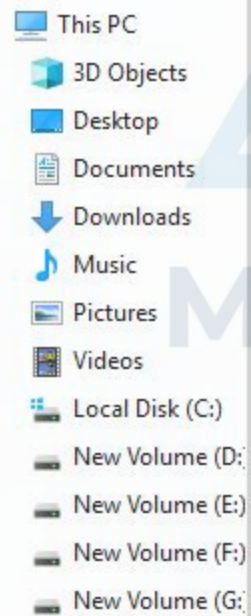
- i. It allows programmers to take advantage of the repetitive structure present in many problems.
- ii. Complex case analysis and nested loops can be avoided.
- iii. Recursion can lead to more readable and efficient algorithm descriptions.

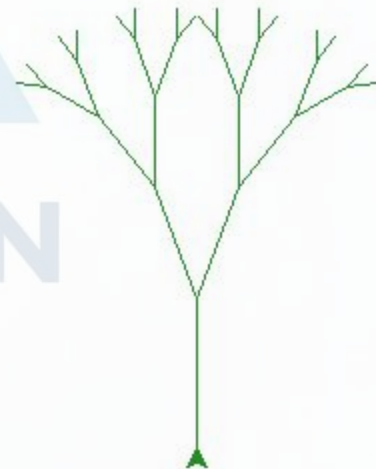
- **Disadvantages:**

- i. Slowing down execution time and storing on the run-time stack.
- ii. There is a danger of running out of space on the stack and ultimately program crashes.
- iii. The run time can be prohibitively long even for very simple cases.

Recursive Functions

- Folders on your file system are directories that can contain subdirectories. A recursive function can be used to process all folders within a folder, and so on.
- Data □ hierarchies, such as grammatical parse trees, organization charts, etc.
- A recursive function can be a convenient structure for processing such hierarchies.

- 
- This PC
 - 3D Objects
 - Desktop
 - Documents
 - Downloads
 - Music
 - Pictures
 - Videos
 - Local Disk (C:)
 - New Volume (D:)
 - New Volume (E:)
 - New Volume (F:)
 - New Volume (G:)



Packages

- Inorder to organize large number of files ☐ based on some criteria, so as to maintain it easily
- MODULE ☐ Multiple objects, such as classes, functions etc.,
- PACKAGE ☐ one or more relevant modules.
- Package in Python takes the concept of the modular approach to next logical level

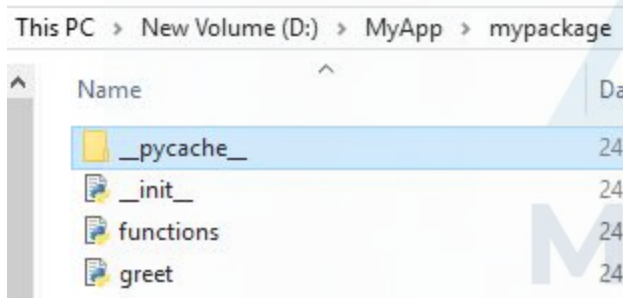
Packages

```
greet.py x  __init__.py x  functions.py x
1 def sum(x,y):
2     return x+y
3
4 def average(x,y):
5     return (x+y)/2
6
7 def power(x,y):
8     return x**y
9
10 def square(x):
11     return x*x
12
13 def fact(n):
14     if n == 0:
15         return 1
16     else:
17         return n * fact(n-1)
18
19 def mycube(x):
20     sqr = x * x;
21     res = sqr * x;
22     return res
```

```
greet.py x  __init__.py x  functions.py x
1 def SayHello(name):
2     print("Hello " + name)
3     return
```

Packages

```
greet.py x  _init_.py x  functions.py x
1  D:\MyApp\mypackage\functions.py: from .functions import average, power, mycube, square, fact, sum
2  D:\MyApp\mypackage\greet.py: from .greet import SayHello
```



```
Microsoft Windows [Version 10.0.17134.48]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\aldo>d:

D:\>cd .

D:\>cd .\MyApp

D:\MyApp>
```

```
Type "help", "copyright", "credits" or "license" for more information.
>>> from mypackage import functions
>>> functions.mycube(3)
27
>>>

>>>
>>> x = print("power(4,2) : ", x)
power(4,2) :  9
>>>
```

OOP concepts in Python

- Python is a multi-paradigm programming language. Meaning, it supports different programming approach.
- The concept of OOP in Python focuses on creating reusable code. This concept is also known as DRY (Don't Repeat Yourself).

- A **class** is a blueprint for the object.
- An **object** (instance) is an instantiation of a class.
- An object has two characteristics:
 1. Attributes
 2. Behavior

Student is an object,

=> name, age, country are attributes

=> singing, dancing are behavior

Creating Class and Object in Python

```
In [50]: class Student:

    # class attribute
    school = "XYZ"

    # instance attribute
    def __init__(self, name, age):
        self.name = name
        self.age = age

# instantiate the Parrot class
Alice = Student("Alice", 10)
Bob = Student("Bob", 15)

# access the class attributes
print("Alice is from {}".format(Alice.__class__.school))
print("Bob is also from {}".format(Bob.__class__.school))

# access the instance attributes
print("{} is {} years old".format( Alice.name, Alice.age))
print("{} is {} years old".format( Bob.name, Bob.age))
```

Alice is from XYZ
Bob is also from XYZ
Alice is 10 years old
Bob is 15 years old

Methods

- Methods are functions defined inside the body of a class.
- //Behaviours//

```
class Student:
    # instance attributes
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # instance method
    def sing(self, song):
        return "{} sings {}".format(self.name, song)

    def dance(self):
        return "{} is now dancing".format(self.name)

# instantiate the object
blu = Student("Blu", 10)

# call our instance methods
print(blu.sing("Happy"))
print(blu.dance())
```

```
Blu sings 'Happy'
Blu is now dancing
```

Inheritance

- Inheritance is a way of creating new class for using details of existing class without modifying it.
- Base class(parent) ☐ derived class(child)

```
In [52]: class Bird:

    def __init__(self):
        print("Bird is ready")

    def whoisThis(self):
        print("Bird")

    def swim(self):
        print("Swim faster")

# child class
class Penguin(Bird):

    def __init__(self):
        # call super() function
        super().__init__()
        print("Penguin is ready")

    def whoisThis(self):
        print("Penguin")

    def run(self):
        print("Run faster")

peggy = Penguin()
peggy.whoisThis()
peggy.swim()
peggy.run()
```

```
Bird is ready
Penguin is ready
Penguin
Swim faster
Run faster
```

Encapsulation

- Restricting access to methods and variables.
- Preventing data from direct modification which is called encapsulation.
- In Python, we denote private attribute using underscore as prefix
i.e single “_” or double “__”.

- The variable is not changeable as it is private

```
In [53]: class Car:
    __maxspeed = 0
    __name = ""

    def __init__(self):
        self.__maxspeed = 200
        self.__name = "Supercar"

    def drive(self):
        print('driving. maxspeed ' + str(self.__maxspeed))

redcar = Car()
redcar.drive()
redcar.__maxspeed = 10 # will not change variable because its private
redcar.drive()

driving. maxspeed 200
driving. maxspeed 200
```

- If you want to change the value of a private variable, a setter method is used. This is simply a method that sets the value of a private variable.

```
In [54]: class Car:
    __maxspeed = 0
    __name = ""

    def __init__(self):
        self.__maxspeed = 200
        self.__name = "Supercar"

    def drive(self):
        print('driving. maxspeed ' + str(self.__maxspeed))

    def setMaxSpeed(self, speed):
        self.__maxspeed = speed

redcar = Car()
redcar.drive()
redcar.setMaxSpeed(320)
redcar.drive()

driving. maxspeed 200
driving. maxspeed 320
```


Polymorphism

- Polymorphism

It's the ability (in OOP) to use common interface for multiple form (data types)

```
class Student_u18:

    def driverlicence(self):
        print("Studnet under 18 can't apply")

    def VoterID(self):
        print("Studnet under 18 can't vote")

class Student_a18:

    def driverlicence(self):
        print("Studnet under 18 can apply")

    def VoterID(self):
        print("Studnet under 18 can vote")

# common interface
def driving_test(candidate):
    candidate.driverlicence()

#instantiate objects
Alice = Student_u18() # Alice is under 18
Bob = Student_a18() #Bob is above 18

# passing the object
driving_test(Alice)
driving_test(Bob)
```

```
Studnet under 18 can't apply
Studnet under 18 can apply
```

Exception Handling in Python

- A piece of code can throw many exception
- The one who code needs to account for all of the potential exceptions that could be raised without creating duplicate code or long, meandering code passages.

try:

except

```
# Python program to handle simple runtime error

a = [1, 2, 3]
try:
    print ("First element = %d" %(a[0]) )

    # Throws error since there are only 3 elements in array
    print ("Fifth element = %d" %(a[4]) )

except IndexError:
    print ("An error occurred")
```

```
First element = 1
An error occurred
```

Python Database connection

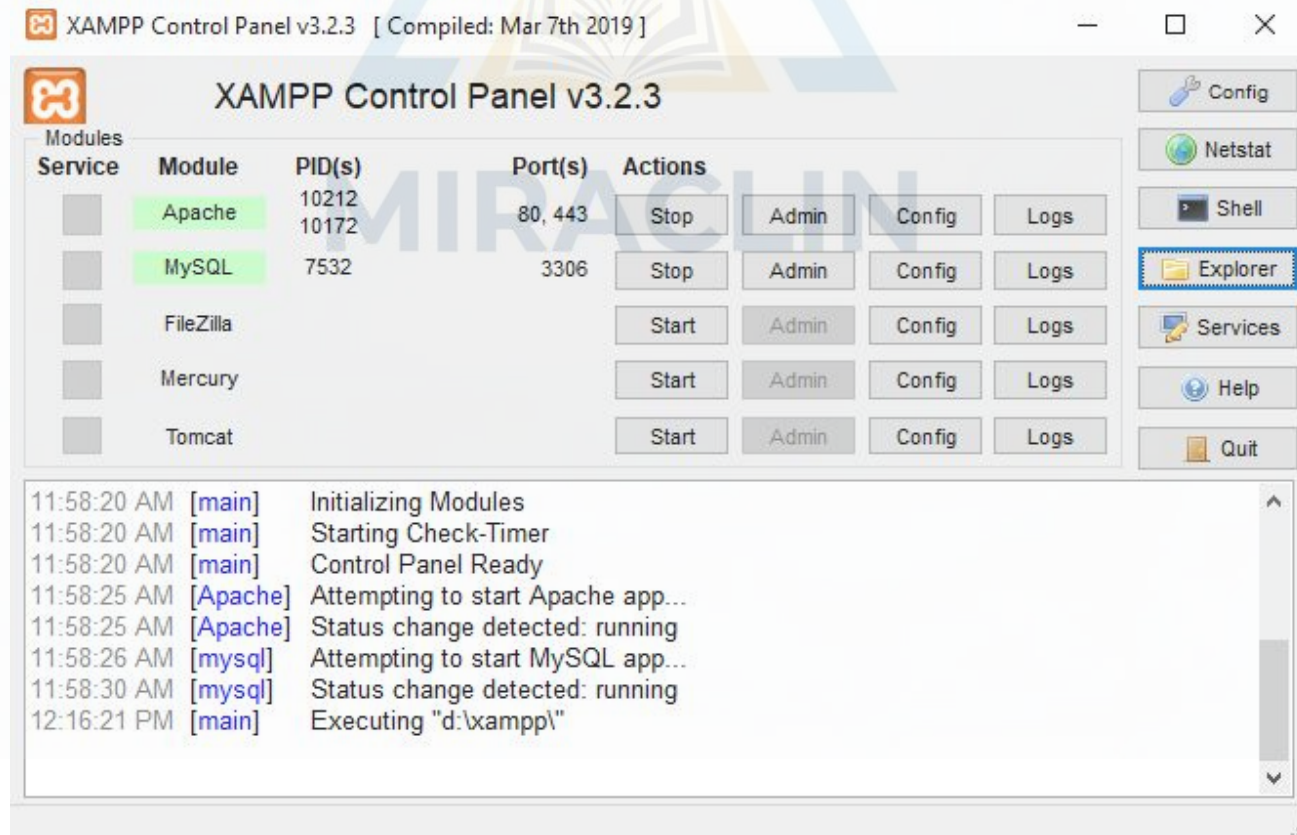
- We can use python to get a connection with the MySQL database.
- Python needs a MySQL driver to access the MySQL database.
- It requires a driver module to get connected.
here we used pymysql.
- `pip install pymysql`

- Testing the module

```
G:\Python\Python37>pip3 install pymysql
Collecting pymysql
  Using cached https://files.pythonhosted.org/packages/ed/39/15045ae46f2a1
5/PyMySQL-0.9.3-py2.py3-none-any.whl
Installing collected packages: pymysql
Successfully installed pymysql-0.9.3

G:\Python\Python37>python
Python 3.7.5 (tags/v3.7.5:5c02a39a0b, Oct 15 2019, 00:11:34) [MSC v.1916 6
Type "help", "copyright", "credits" or "license" for more information.
>>> import pymysql.cursors
>>>
```

- Make sure that the MySQL database is ready,
- Here we used XAMPP, and started the MySQL.



Creating the Connection

```
In [1]: import pymysql
# Connect to the database
connection = pymysql.connect(host='localhost',
                             user='root',
                             password='',
                             db='db1',
                             charset='utf8mb4',
                             cursorclass=pymysql.cursors.DictCursor)

cursor = connection.cursor()

# execute SQL query using execute() method.
cursor.execute("SELECT VERSION()")

# Fetch a single row using fetchone() method.
data = cursor.fetchone()
print ("Database version : %s " % data)

# disconnect from server
connection.close()
```

Database version : {'VERSION()': '10.1.38-MariaDB'}

Check if Database Exists

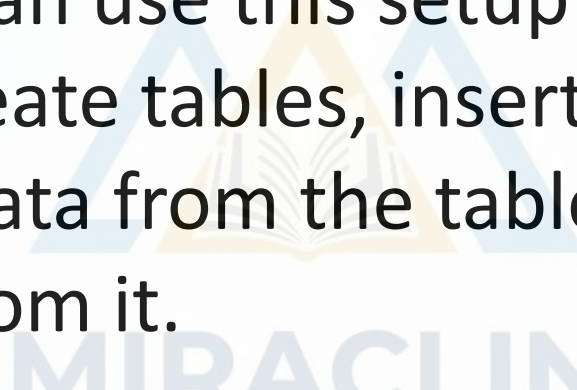
```
In [3]: import pymysql

mydb = pymysql.connect(
    host="localhost",
    user="root",
    passwd=""
)

mycursor = mydb.cursor()
mycursor.execute("SHOW DATABASES")

for x in mycursor:
    print(x)

('db1',)
('db2',)
('db3',)
('information_schema',)
('mvc',)
('mysql',)
('performance_schema',)
('phpmyadmin',)
('test',)
```


- 
- The logo for MiracLIN is centered in the background. It features a stylized yellow triangle with an open book icon inside it. Below the triangle, the word "MIRACLIN" is written in a light blue, sans-serif font.
- Similarly we can use this setup to create databases, create tables, insert data into the tables, read data from the tables, update and delete data from it.
 - Thus performing the CRUD operations.

Regular expression

- A *regular expression* is a special sequence
- Characters that helps you match/find other strings or sets of strings
- It uses a specialized syntax held in a pattern.
- Mostly found in UNIX
- It is extremely useful for extracting information from text such as code, files, log, spreadsheets or even documents

Regular Expression Syntax

- `import re`
- "re" module included with Python primarily used for string searching and manipulation
- Also used frequently for web page "Scraping" (extract large amount of data from websites)
- The method we going to see are
- `re.match()`, `re.search()`, `re.findall()`

re.match()

```
In [16]: #match
import re
list = ["aldrin all", "bob", "alice all", "Smith", "aldrin and alice are here", "alice and bob are buddies"]
for element in list:
    z = re.match("(a\w+)\W(a\w+)", element)
    if z:
        print((z.groups()))

('aldrin', 'all')
('alice', 'all')
('aldrin', 'and')
('alice', 'and')
```

re.search()

```
#Search
import re
patterns = ['python scripting', 'playing']
text = 'What is python scripting?'

for pattern in patterns:
    print('Looking for "%s" in "%s"-->' %(pattern,text), end='')
    if re.search(pattern, text):
        print('found a match')
    else:
        print('no match')
```

Looking for "python scripting" in "What is python scripting?"-->found a match
Looking for "playing" in "What is python scripting?"-->no match

re.findall()

```
#findall
import re

a = 'aldrin@123.com, aldrin@321.com,user1@123.com,root@localhost.com'
emails = re.findall(r'[\w\.-]+@[\w\.-]+', a)
for email in emails:
    print(email)
```

```
aldrin@123.com
aldrin@321.com
user1@123.com
root@localhost.com
```

re.split()

```
#split  
import re  
print(re.split(r'\s', 'we are splitting the words'))  
  
['we', 'are', 'splitting', 'the', 'words']  
  
print((re.split(r's', 'split the words')))  
  
['', 'plit the word', '']
```

- Let's try to find phone numbers.

```
def is_phone_number(text):  
    '''Return True if `text` is a valid US phone number.  
  
    A phone number in the US is a string of digits with a 3-digit Area Code,  
    followed by hyphen, a group of three digits, another hyphen and a group  
    of four digits.'''  
    # Test the length of the string  
    if len(text) != 12:  
        return False  
    # Test that the first three characters are digits  
    for i in range(3):  
        if not text[i].isnumeric():  
            return False  
    # Test that the fourth character is a '-'  
    if text[3] != '-':  
        return False  
    # Test that the next three characters are digits  
    for i in range(4,7):  
        if not text[i].isnumeric():  
            return False  
    # Test that the next character is a '-'  
    if text[7] != '-':  
        return False  
    # Test that the last four characters are digits  
    for i in range(8,12):  
        if not text[i].isnumeric():  
            return False  
    # If we didn't fail any of the above tests, it's a valid US phone number  
    return True
```



```
test_string = '510-654-1220'  
is_phone_number(test_string)
```

True

```
test_string = 'this is not a phone number'  
is_phone_number(test_string)
```

False

```
test_string = '415-677-211' # missing final digit  
is_phone_number(test_string)
```

False

```
message = 'Call me at 409-223-8952 tomorrow. 409-888-8498 is my office'  
for i in range (len(message)):  
    chunk = message[i:i+12]  
    if is_phone_number(chunk):  
        print('Phone number found: ' + chunk)
```

Phone number found: 409-223-8952

Phone number found: 409-888-8498

```
import re  
phone_number_pattern = '\d{3}-\d{3}-\d{4}'  
for number in re.findall(phone_number_pattern, message):  
    print('Phone number found: ' + number)
```

Phone number found: 409-223-8952

Phone number found: 409-888-8498

What is a Multithreading ?

- Multithreading is a technique which allows a CPU to execute multiple threads at the same time. These threads can execute individually while sharing their process resources.
- The difference is shown in the following table

S.NO	MULTIPROCESSING	MULTITHREADING
1.	In Multiprocessing, CPUs are added for increasing computing power.	While In Multithreading, many threads are created of a single process for increasing computing power.
2.	In Multiprocessing, Many processes are executed simultaneously.	While in multithreading, many threads of a process are executed simultaneously.
3.	Multiprocessing are classified into Symmetric and Asymmetric .	While Multithreading is not classified in any categories.
4.	In Multiprocessing, Process creation is a time-consuming process.	While in Multithreading, process creation is according to economical.

Python Multithreading

Multithreading in Python with Global Interpreter Lock(GIL)

- Syntax to create a new thread
- *thread.start_new_thread(function_name, arguments)*
- accessible as **__thread**

```
import time
import _thread

def thread_test(name, wait):
    i = 0
    while i <= 3:
        time.sleep(wait)
        print("Running %s\n" %name)
        i = i + 1

    print("%s has finished execution" %name)

if __name__ == "__main__":
    _thread.start_new_thread(thread_test, ("First Thread", 1))
    _thread.start_new_thread(thread_test, ("Second Thread", 2))
    _thread.start_new_thread(thread_test, ("Third Thread", 3))
```

The Thread Module

Running First Thread

Running Second Thread

Running First Thread

Running Third Thread

Running First Thread

RACE CONDITIONS

Running Second Thread

Running First Thread

First Thread has finished execution

Running Second Thread

Running Third Thread

Running Second Thread

Second Thread has finished execution

Running Third Thread

Running Third Thread

Third Thread has finished execution

The Threading Module

- High-level implementation of threading in python
- RACE CONDITIONS: It is an unwanted state of a program which occurs when a system performs two or more operations simultaneously.

Running First Thread

Running Second Thread

Running First Thread

Running Third Thread

Running First Thread

Running Second Thread

Running First Thread

Lock()

```
import threading
lock = threading.Lock()

def first_function():
    for i in range(5):
        lock.acquire()
        print ('lock acquired')
        print ('Executing the first function')
        lock.release()

def second_function():
    for i in range(5):
        lock.acquire()
        print ('lock acquired')
        print ('Executing the second function')
        lock.release()

if __name__=="__main__":
    thread_one = threading.Thread(target=first_function)
    thread_two = threading.Thread(target=second_function)

    thread_one.start()
    thread_two.start()

    thread_one.join()
    thread_two.join()
```

```
lock acquired
Executing the first funcnion
lock acquired
Executing the first funcnion
lock acquired
Executing the first funcnion
lock acquired
Executing the first funcnion
lock acquired
Executing the first funcnion
lock acquired
Executing the second funcnion
lock acquired
Executing the second funcnion
lock acquired
Executing the second funcnion
lock acquired
Executing the second funcnion
lock acquired
Executing the second funcnion
```

NumPy for mathematical computing

- Various mathematical operations
- Trigonometric functions, functions for arithmetic operations, handling complex numbers, etc
- `import numpy as np`
 - `np.sin()`
 - `np.cos()`
 - `np.cosh()`

```
In [4]: math.pi
```

```
Out[4]: 3.141592653589793
```

```
In [5]: np.pi
```

```
Out[5]: 3.141592653589793
```

```
In [6]: np.sin
```

```
Out[6]: <ufunc 'sin'>
```



```
In [1]: import numpy as np
import math
```

```
In [2]: in_array = [0, math.pi / 2, np.pi / 3, np.pi]
print ("Input array : \n", in_array)
```

Input array :
[0, 1.5707963267948966, 1.0471975511965976, 3.141592653589793]

```
In [3]: Sin_Values = np.sin(in_array)
print ("\nSine values : \n", Sin_Values)
```

Sine values :
[0.00000000e+00 1.00000000e+00 8.66025404e-01 1.22464680e-16]

```
In [7]: Sinh_Values = np.sinh(in_array)
print ("\nSine Hyperbolic values : \n", Sinh_Values)
```

Sine Hyperbolic values :
[0. 2.3012989 1.24936705 11.54873936]

- **`tanh()`**: Compute hyperbolic tangent element-wise.
- **`arcsinh()`**: Inverse hyperbolic sine element-wise.
- **`arccosh()`**: Inverse hyperbolic cosine, element-wise.
- **`arctanh()`**: Inverse hyperbolic tangent element-wise.

Functions for Rounding

```
import numpy as np  
np.around(*inputArray*)
```

```
In [8]: in_array = [.5, 1.5, 2.5, 3.5, 4.5, 10.1]  
print ("Input array : \n", in_array)  
  
round_off_values = np.around(in_array)  
print ("\nRounded values : \n", round_off_values)
```

```
Input array :  
[0.5, 1.5, 2.5, 3.5, 4.5, 10.1]
```

```
Rounded values :  
[ 0.  2.  2.  4.  4. 10.]
```

- `np.round_(in_array)`

```
In [9]: in_array = [.5, 1.5, 2.5, 3.5, 4.5, 10.1]
print ("Input array : \n", in_array)

round_off_values = np.round_(in_array)
print ("\nRounded values : \n", round_off_values)
```

```
Input array :
[0.5, 1.5, 2.5, 3.5, 4.5, 10.1]
```

```
Rounded values :
[ 0.  2.  2.  4.  4. 10.]
```

FUNCTION	DESCRIPTION
rint()	Round to nearest integer towards zero.
fix()	Round to nearest integer towards zero.
floor()	Return the floor of the input, element-wise.
ceil()	Return the ceiling of the input, element-wise.
trunc()	Return the truncated value of the input, element-wise.

Exponents and logarithms Functions

- `np.exp()`

```
In [10]: in_array = [1, 3, 5]
print ("Input array : ", in_array)

out_array = np.exp(in_array)
print ("Output array : ", out_array)

Input array : [1, 3, 5]
Output array : [ 2.71828183 20.08553692 148.4131591 ]
```

- `np.log()`

```
In [11]: in_array = [1, 3, 5, 2**8]
print ("Input array : ", in_array)

out_array = np.log(in_array)
print ("Output array : ", out_array)

Input array : [1, 3, 5, 256]
Output array : [0.          1.09861229 1.60943791 5.54517744]
```

- **log10()**: Return the base 10 logarithm of the input array, element-wise.
- **log2()**: Base-2 logarithm of x.
- **log1p()**: Return the natural logarithm of one plus the input array, element-wise.
- **logaddexp()**: Logarithm of the sum of exponentiations of the inputs.
- **logaddexp2()**: Logarithm of the sum of exponentiations of the inputs in base-2.

- `np.reciprocal()`

```
In [12]: in_num = 2.0
print ("Input number : ", in_num)

out_num = np.reciprocal(in_num)
print ("Output number : ", out_num)

Input number : 2.0
Output number : 0.5
```

- `np.divide()`

```
In [13]: # input_array
arr1 = [2, 27, 2, 21, 23]
arr2 = [2, 3, 4, 5, 6]
print ("arr1      : ", arr1)
print ("arr2      : ", arr2)

# output_array
out = np.divide(arr1, arr2)
print ("\nOutput array : \n", out)

arr1      : [2, 27, 2, 21, 23]
arr2      : [2, 3, 4, 5, 6]

Output array :
[1.      9.      0.5      4.2      3.83333333]
```

- **mod()**: Return the element-wise remainder of division.
- **remainder()**: Return element-wise remainder of division.
- **divmod()**: Return element-wise quotient and remainder simultaneously.
- **sqrt()**: Return the non-negative square-root of an array, element-wise.
- **square()**: Return the element-wise square of the input.

- **cbrt ()** function

```
In [14]: arr1 = [1, 27000, 64, -1000]
print ("cbrt Value of arr1 : \n", np.cbrt(arr1))

cbrt Value of arr1 :
[ 1. 30.  4. -10.]
```

- **clip()** function

```
In [15]: in_array = [1, 2, 3, 4, 5, 6, 7, 8 ]
print ("Input array : ", in_array)

out_array = np.clip(in_array, a_min = 2, a_max = 6)
print ("Output array : ", out_array)

Input array : [1, 2, 3, 4, 5, 6, 7, 8]
Output array : [2 2 3 4 5 6 6 6]
```

- **maximum()**: Element-wise maximum of array elements.
- **minimum()**: Element-wise minimum of array elements.

Matplotlib for data visualization

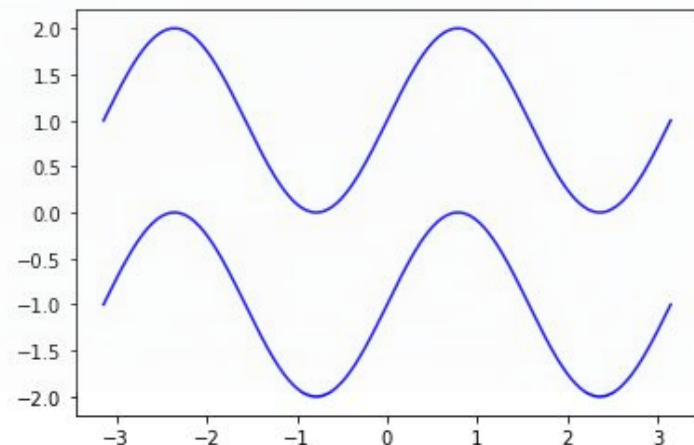
- *Regular Plots*

import numpy as np

import matplotlib.pyplot as plt

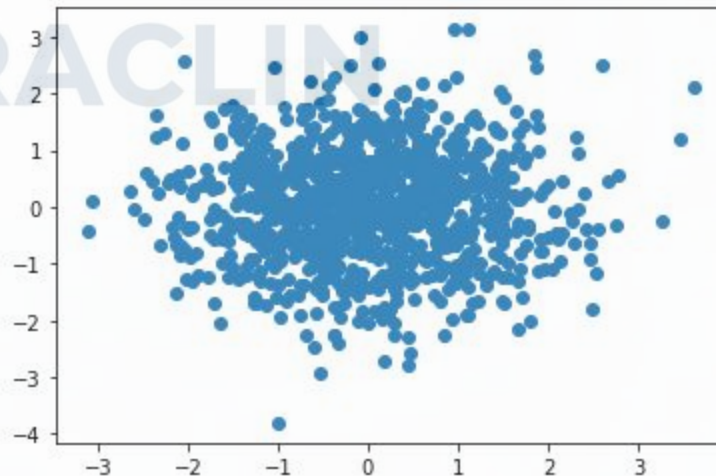
plt.show()

```
In [19]: n = 256  
X = np.linspace(-np.pi, np.pi, n, endpoint=True)  
Y = np.sin(2 * X)  
plt.plot(X, Y + 1, color='blue', alpha=1.00)  
plt.plot(X, Y - 1, color='blue', alpha=1.00)  
plt.show()
```



- *Scatter Plots*
plt.scatter(X,Y)
plt.show()

```
In [20]: n = 1024  
X = np.random.normal(0,1,n)  
Y = np.random.normal(0,1,n)  
plt.scatter(X,Y)  
plt.show()
```

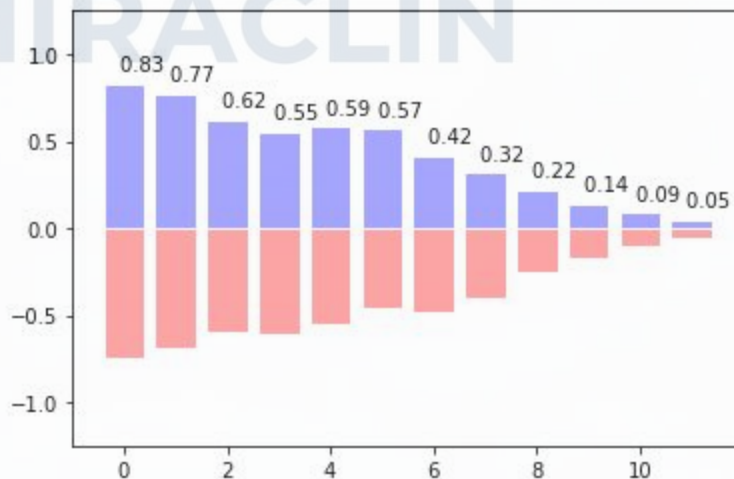


- *Bar Plots*

plt.bar()

plt.show()

```
In [21]: n = 12
X = np.arange(n)
Y1 = (1 - X / float(n)) * np.random.uniform(0.5, 1.0, n)
Y2 = (1 - X / float(n)) * np.random.uniform(0.5, 1.0, n)
plt.bar(X, +Y1, facecolor='#9999ff', edgecolor='white')
plt.bar(X, -Y2, facecolor='#ff9999', edgecolor='white')
for x, y in zip(X, Y1):
    plt.text(x + 0.4, y + 0.05, '%.2f ' % y, ha='center', va='bottom')
plt.ylim(-1.25, +1.25)
plt.show()
```

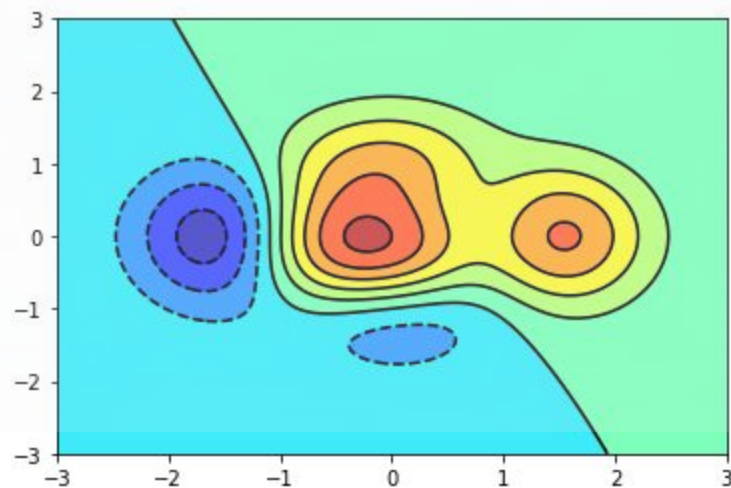


- *Contour Plots*

plt.contour()

```
In [26]: def f(x, y):  
          return (1 - x / 2 + x ** 5 + y ** 3) * np.exp(-x ** 2 - y ** 2)  
          n = 256  
          x = np.linspace(-3, 3, n)  
          y = np.linspace(-3, 3, n)  
          X, Y = np.meshgrid(x, y)  
          plt.contourf(X, Y, f(X, Y), 8, alpha=.75, cmap='jet')  
          C = plt.contour(X, Y, f(X, Y), 8, colors='black', linewidth=.5)
```

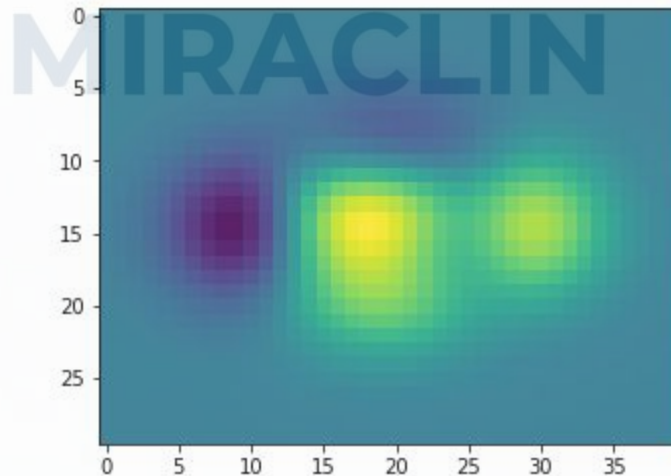
D:\Anaconda3\lib\site-packages\ipykernel_launcher.py:8: UserWarning: The



- *Imshow*

plt.imshow()

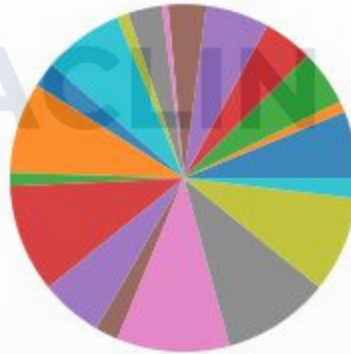
```
In [27]: def f(x, y):  
          return (1 - x / 2 + x ** 5 + y ** 3) * np.exp(-x ** 2 - y ** 2)  
          n = 10  
          x = np.linspace(-3, 3, 4 * n)  
          y = np.linspace(-3, 3, 3 * n)  
          X, Y = np.meshgrid(x, y)  
          plt.imshow(f(X, Y))  
          plt.show()
```



- *pie charts*

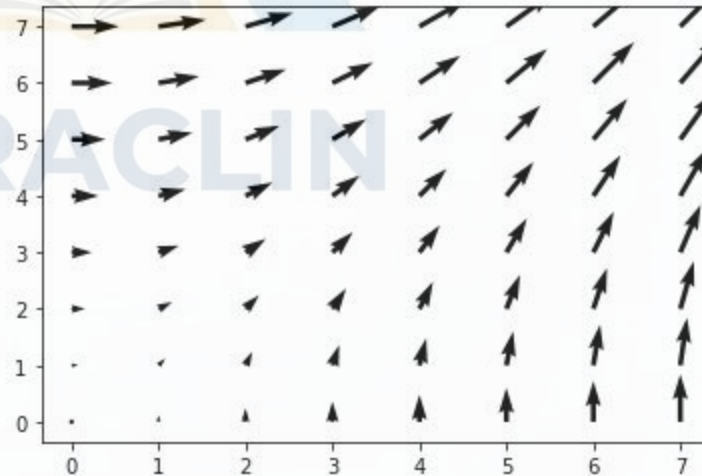
plt.pie()

```
In [28]: Z = np.random.uniform(0, 1, 20)  
plt.pie(Z)  
plt.show()
```



- *Quiver Plots*
plt.quiver()

```
In [29]: n = 8  
X, Y = np.mgrid[0:n, 0:n]  
plt.quiver(X, Y)  
plt.show()
```



- *Grids*
plt.grid()

```
In [30]: # The Data
x = [1, 2, 3, 4]
y = [234, 124, 368, 343]

# Create the figure and axes objects
fig, ax = plt.subplots(1, figsize=(8, 6))
fig.suptitle('Example Of Plot With Grid Lines')

# Plot the data
ax.plot(x,y)

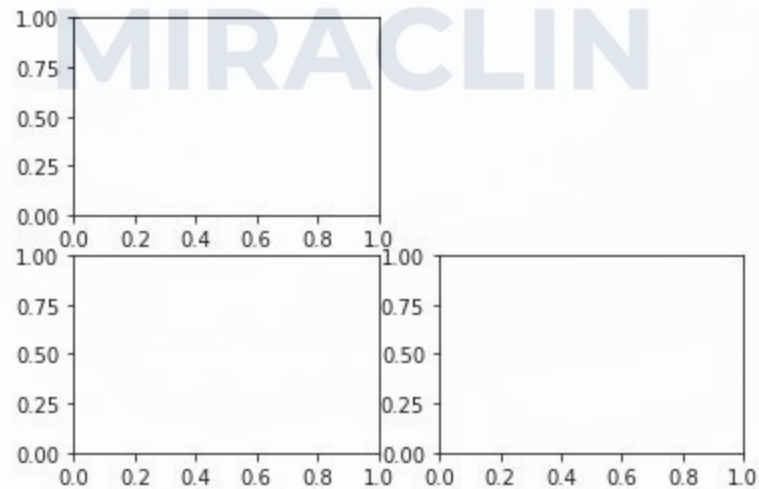
# Show the grid lines as dark grey lines
plt.grid(b=True, which='major', color='#666666', linestyle='-')

plt.show()
```



- *Multi Plots*

```
In [31]: plt.subplot(2, 2, 1)  
plt.subplot(2, 2, 3)  
plt.subplot(2, 2, 4)  
plt.show()
```

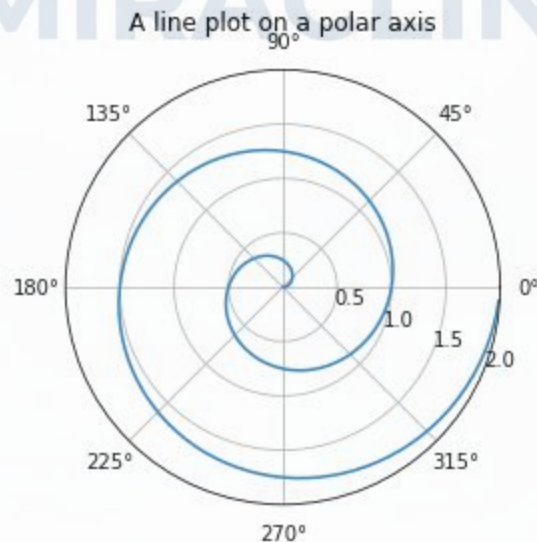


- *Polar Axis*

```
In [32]: r = np.arange(0, 2, 0.01)
theta = 2 * np.pi * r

ax = plt.subplot(111, projection='polar')
ax.plot(theta, r)
ax.set_rmax(2)
ax.set_rticks([0.5, 1, 1.5, 2]) # less radial ticks
ax.set_rlabel_position(-22.5) # get radial labels away from plotted line
ax.grid(True)

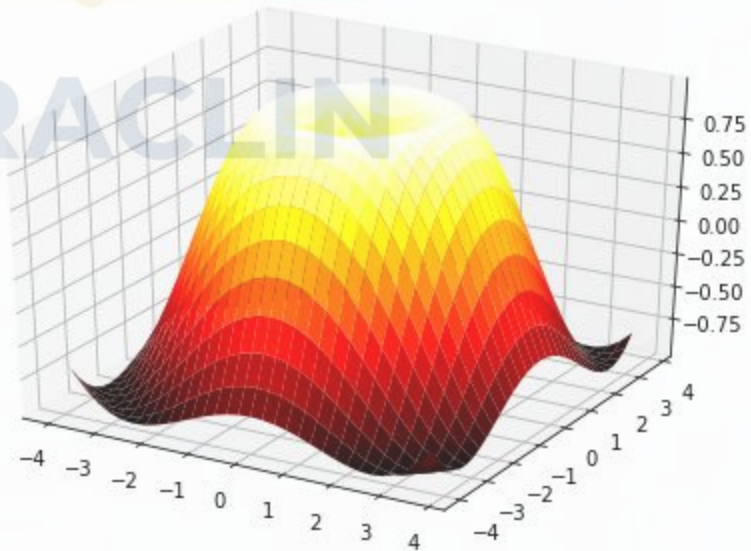
ax.set_title("A line plot on a polar axis", va='bottom')
plt.show()
```



- *3D Plots*

```
In [33]: from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = Axes3D(fig)
X = np.arange(-4, 4, 0.25)
Y = np.arange(-4, 4, 0.25)
X, Y = np.meshgrid(X, Y)
R = np.sqrt(X**2 + Y**2)
Z = np.sin(R)
ax.plot_surface(X, Y, Z, rstride=1, cstride=1, cmap='hot')
```

```
Out[33]: <mpl_toolkits.mplot3d.art3d.Poly3DCollection at 0x1bd8c6d2c08>
```



Scipy

- Scipy, I/O package, has a wide range of functions for work with different files format which are Matlab, Arff, Wave, Matrix Market, IDL, NetCDF, TXT, CSV and binary format.
- SciPy is built in top of the NumPy
- SciPy is a fully-featured version of Linear Algebra while Numpy contains only a few features.
- Most new Data Science features are available in Scipy rather than Numpy.

- Cube root from scipy.
- ```
from scipy.special import cbrt
cb = cbrt([a, b])
```

```
In [34]: from scipy import special #same for other modules
import numpy as np
```

```
In [36]: from scipy.special import cbrt
 #Find cubic root of 27 & 64 using cbrt() function
 cb = cbrt([27, 64])
 #print value of cb
 print(cb)
```

```
[3. 4.]
```

- Exponential function
- `from scipy.special import exp10`
- Exponential function computes the  $10^{**}x$  element-wise.

```
In [37]: from scipy.special import exp10
 #define exp10 function and pass value in its
 exp = exp10([1,10])
 print(exp)

 [1.e+01 1.e+10]
```

- Permutations & Combinations
- **Combinations** - `scipy.special.comb(N,k)`
- **Permutations** - `scipy.special.perm(N,k)`

```
In [38]: from scipy.special import comb
#find combinations of 5, 2 values using comb(N, k)
com = comb(5, 2, exact = False, repetition=True)
print(com)
```

15.0

```
In [39]: from scipy.special import perm
#find permutation of 5, 2 using perm (N, k) function
per = perm(5, 2, exact = True)
print(per)
```

20



- Linear Algebra with SciPy
- **Determinant** of a two-dimensional matrix
- `linalg.det( )`

---

```
In [40]: from scipy import linalg
import numpy as np
#define square matrix
two_d_array = np.array([[4,5], [3,2]])
#pass values to det() function
linalg.det(two_d_array)
```

```
Out[40]: -7.0
```

---

- **Inverse Matrix :** Inverse Matrix of Scipy calculates the inverse of any square matrix.
- `scipy.linalg.inv()`

```
In [41]: from scipy import linalg
import numpy as np
define square matrix
two_d_array = np.array([[4,5], [3,2]])
#pass value to function inv()
linalg.inv(two_d_array)

Out[41]: array([[-0.28571429, 0.71428571],
 [0.42857143, -0.57142857]])
```

- **Eigenvalues and Eigenvector**
  - `scipy.linalg.eig()`
- The most common problem in linear algebra is eigenvalues and eigenvector which can be easily solved using **`eig()`** function.

```
In [42]: #define two dimensional array
arr = np.array([[5,4],[6,3]])
#pass value into function
eg_val, eg_vect = linalg.eig(arr)
#get eigenvalues
print(eg_val)
#get eigenvectors
print(eg_vect)
```

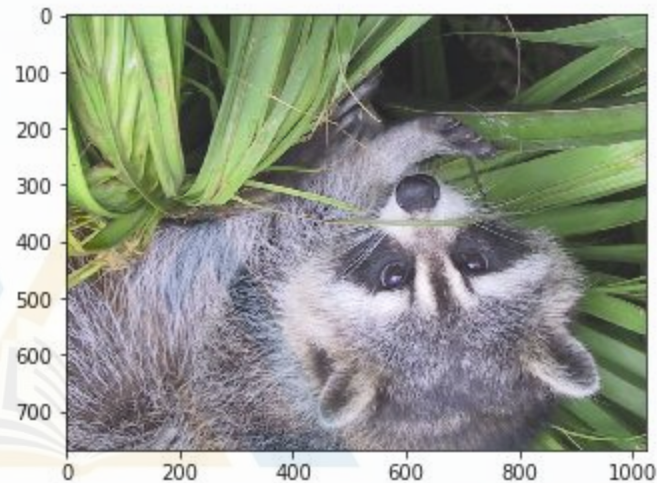
```
[9.+0.j -1.+0.j]
[[0.70710678 -0.5547002]
 [0.70710678 0.83205029]]
```

- **Image Processing with SciPy – `scipy.ndimage`**
- **MISC Package** in SciPy contains prebuilt images which can be used to perform image manipulation task
- `scipy.ndimage` is a submodule of SciPy which is mostly used for performing an image related operation
- `ndimage` means the "n" dimensional image.
- SciPy Image Processing provides Geometrics transformation (rotate, crop, flip), image filtering (sharp and de nosing), display image, image segmentation, classification and features extraction.

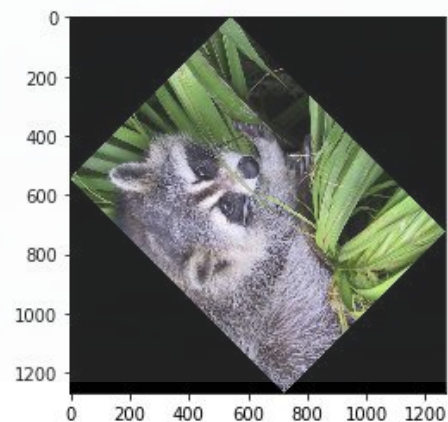
```
from scipy import misc
from matplotlib import pyplot as plt
import numpy as np
#get face image of panda from misc package
panda = misc.face()
#plot or show image of face
plt.imshow(panda)
plt.show()
```



```
#Flip Down using scipy misc.face image
flip_down = np.flipud(misc.face())
plt.imshow(flip_down)
plt.show()
```



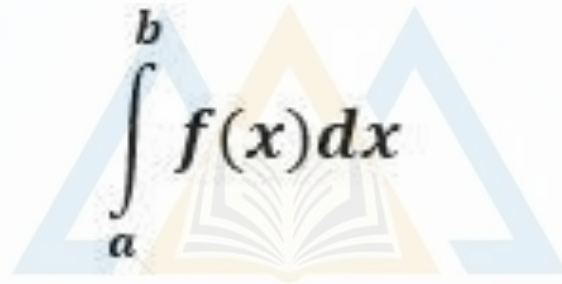
```
from scipy import ndimage, misc
from matplotlib import pyplot as plt
panda = misc.face()
#rotation function of scipy for image - image rotated 135 degree
panda_rotate = ndimage.rotate(panda, 135)
plt.imshow(panda_rotate)
plt.show()
```



- **Integration with Scipy – Numerical Integration**
- **scipy.integrate** library has single integration, double, triple, multiple, Gaussian quadrature, Romberg, Trapezoidal and Simpson's rules.

$$\int_a^b f(x)dx$$





# MIRACLIN

```
In [47]: from scipy import integrate
take f(x) function as f
f = lambda x : x**2
#single integration with a = 0 & b = 1
integration = integrate.quad(f, 0 , 1)
print(integration)

(0.33333333333333337, 3.700743415417189e-15)
```

# Pandas

- We can analyze data in pandas with:

**Series**

**DataFrames**



# Series

```
Program to Create series with scalar values
Data =[1, 3, 4, 5, 6, 2, 9] # Numeric data

Creating series with default index values
s = pd.Series(Data)

predefined index values
Index =['a', 'b', 'c', 'd', 'e', 'f', 'g']

Creating series with predefined index values
si = pd.Series(Data, Index)
```

```
Program to Create Dictionary series
dictionary ={'a':1, 'b':2, 'c':3, 'd':4, 'e':5}

Creating series of Dictionary type
sd = pd.Series(dictionary)
sd
```

```
a 1
b 2
c 3
d 4
e 5
dtype: int64
```

```
s
0 1
1 3
2 4
3 5
4 6
5 2
6 9
dtype: int64
```

```
si
a 1
b 3
c 4
d 5
e 6
f 2
g 9
dtype: int64
```

```
Program to Create ndarray series
Data =[[2, 3, 4], [5, 6, 7]] # Defining 2darray

Creating series of 2darray
snd = pd.Series(Data)
snd
```

```
0 [2, 3, 4]
1 [5, 6, 7]
dtype: object
```

# DataFrame

```
Program to Create Data Frame with two dictionaries
dict1 ={'a':1, 'b':2, 'c':3, 'd':4} # Define Dictionary 1
dict2 ={'a':5, 'b':6, 'c':7, 'd':8, 'e':9} # Define Dictionary 2
Data = {'first':dict1, 'second':dict2} # Define Data with dict1 and dict2
df = pd.DataFrame(Data) # Create DataFrame
df
```

|   | first | second |
|---|-------|--------|
| a | 1.0   | 5      |
| b | 2.0   | 6      |
| c | 3.0   | 7      |
| d | 4.0   | 8      |
| e | NaN   | 9      |

```

Program to create Dataframe of three series
import pandas as pd

s1 = pd.Series([1, 3, 4, 5, 6, 2, 9]) # Define series 1
s2 = pd.Series([1.1, 3.5, 4.7, 5.8, 2.9, 9.3]) # Define series 2
s3 = pd.Series(['a', 'b', 'c', 'd', 'e']) # Define series 3

Data = {'first':s1, 'second':s2, 'third':s3} # Define Data
dfseries = pd.DataFrame(Data) # Create DataFrame
dfseries

```

|   | first | second | third |
|---|-------|--------|-------|
| 0 | 1     | 1.1    | a     |
| 1 | 3     | 3.5    | b     |
| 2 | 4     | 4.7    | c     |
| 3 | 5     | 5.8    | d     |
| 4 | 6     | 2.9    | e     |
| 5 | 2     | 9.3    | NaN   |
| 6 | 9     | NaN    | NaN   |

```
Program to create DataFrame from 2D array
import pandas as pd # Import Library
d1 = [[2, 3, 4], [5, 6, 7]] # Define 2d array 1
d2 = [[2, 4, 8], [1, 3, 9]] # Define 2d array 2
Data = {'first': d1, 'second': d2} # Define Data
df2d = pd.DataFrame(Data) # Create DataFrame
df2d
```

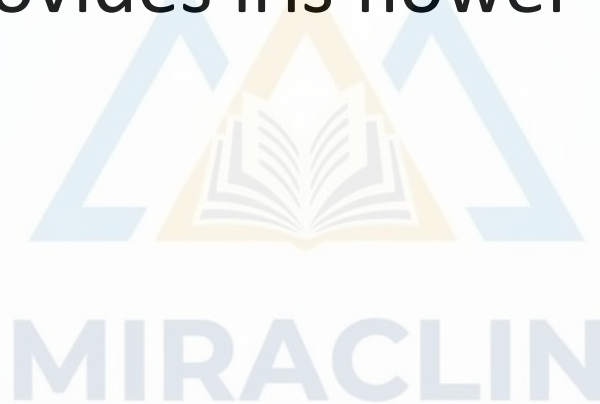
|   | first     | second    |
|---|-----------|-----------|
| 0 | [2, 3, 4] | [2, 4, 8] |
| 1 | [5, 6, 7] | [1, 3, 9] |

# Simple Machine learning



# Data visualization using pandas and classify iris species

- Scikit-learn provides iris flower dataset
1. Visualization
  2. Classification





- Loading dataset

```
from sklearn import datasets
#loading dataset
iris = datasets.load_iris()
#printing feature names
print('features: %s'%iris['feature_names'])
#printing species of iris
print('target categories: %s'%iris['target_names'])
#iris data shape
print("Shape of data: {}".format(iris['data'].shape))
#print data sample
print("sample of data: \n{}".format(iris['data'][:5]))
```

```
features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
target categories: ['setosa' 'versicolor' 'virginica']
Shape of data: (150, 4)
sample of data:
[[5.1 3.5 1.4 0.2]
 [4.9 3. 1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5. 3.6 1.4 0.2]]
```

- We can see there are four features given for each flower are sepal length, sepal width, petal length, petal width and three species of iris are setosa, versicolor and virginica.
- There are 150 total rows/data points which means each species is having 50 samples.
- We could see sample of array in the end of the output where each row has 4 columns.



# KNN

```
#KNN
import numpy as np
from sklearn import datasets
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

#loading dataset
iris = datasets.load_iris()
#splliting data for testing and training
x_train, x_test, y_train, y_test = train_test_split(iris['data'], iris['target'])
#initializing knn
knn = KNeighborsClassifier(n_neighbors=5)
#training
knn.fit(x_train, y_train)
#testing
predicted_specis = knn.predict(x_test)
print("predicted data\n{}".format(predicted_specis))
#printing % of accuracy using mean
print("accuracy: {:.2f}%".format(100*np.mean(predicted_specis == y_test)))
```

predicted data

```
[0 1 2 0 1 2 0 1 0 2 1 0 1 1 2 0 2 0 2 2 0 2 0 1 2 1 0 0 2 2 2 0 2 1 1 1 2
 2]
```

accuracy: 97.37%

# SVM

```
#SVC
import numpy as np
from sklearn import datasets
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split

#loading dataset
iris = datasets.load_iris()
#splliting data for testing and training
x_train, x_test, y_train, y_test = train_test_split(iris['data'], iris['target'])
#initializing knn
svm = SVC(kernel='rbf', random_state=0, gamma=.10, C=1.0)
#training
svm.fit(x_train, y_train)
#testing
predicted_specis = svm.predict(x_test)
print("predicted data\n{}".format(predicted_specis))
#printing % of accuracy using mean
print("accuracy: {:.2f}%".format(100*np.mean(predicted_specis == y_test)))

predicted data
[2 2 0 2 1 1 2 0 1 0 2 2 1 1 0 0 2 1 2 0 2 0 1 1 2 2 1 2 0 0 2 1 0 0 1 1 1
 0]
accuracy: 100.00%
```

# Decision Tree

```
#Decision tree
import numpy as np
from sklearn import datasets
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn import tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

#Loading dataset
iris = datasets.load_iris()
#splliting data for testing and training
x_train, x_test, y_train, y_test = train_test_split(iris['data'], iris['target'])
#initializing knn
ran= RandomForestClassifier()
#training
ran.fit(x_train, y_train)
#testing
predicted_specis = ran.predict(x_test)
print("predicted data\n{}".format(predicted_specis))
#printing % of accuracy using mean
print("accuracy: {:.2f}%".format(100*np.mean(predicted_specis == y_test)))
```

predicted data

```
[2 0 1 2 2 1 0 2 0 2 0 0 0 2 1 0 1 2 1 0 0 0 1 2 1 1 1 2 2 1 2 0 0 0 2 1 1
 0]
```

accuracy: 89.47%

# Random forest

```
#Random Forest
import numpy as np
from sklearn import datasets
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn import tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

#loading dataset
iris = datasets.load_iris()
#splliting data for testing and training
x_train, x_test, y_train, y_test = train_test_split(iris['data'], iris['target'])
#initializing knn
ran= RandomForestClassifier()
#training
ran.fit(x_train, y_train)
#testing
predicted_specis = ran.predict(x_test)
print("predicted data\n{}".format(predicted_specis))
#printing % of accuracy using mean
print("accuracy: {:.2f}%".format(100*np.mean(predicted_specis == y_test)))
```

predicted data

```
[2 1 1 0 0 0 2 2 2 1 2 2 2 0 2 2 0 1 1 0 2 0 1 0 0 2 0 2 0 1 0 1 0 2 0 2 2
 1]
```

accuracy: 94.74%

- *train\_test\_split* method split data for training and testing, 75% data means 112 rows out of 150 will be used to train the algorithm and 25% data or 38 rows will be used to test.
- *fit* method is used to train the classifier and *predict* method is used to classify new items.
- *predict* method will take array of test data and returns predicted species in array format. 0,1,2 are setosa, versicolor and virginica respectively.
- We can calculate accuracy by calculating mean between test and predicted data.